Aula 32 — Docker: Conceitos e Arquitetura

Computação em Nuvem e Virtualização

Ricardo Pasquati Pontarolli

Técnico em Redes de Computadores · IFSP Boituva · pasquati@ifsp.edu.br · v44

Objetivos da Aula

- Compreender a **arquitetura do Docker**.
- Diferenciar **imagem, contêiner e registro**.
- Entender o **Dockerfile** e as camadas.
- Conhecer os principais **comandos**.

O que é Docker?

- Plataforma para **criar, distribuir e executar** contêineres.
- Facilita empacotar aplicações com suas dependências.
- Padroniza o ambiente entre desenvolvimento e produção.
- Base de boa parte dos fluxos de **DevOps**.

Arquitetura do Docker

- **Cliente (CLI):** `docker`.
- **Daemon (`dockerd`):** executa contêineres.
- **Registro:** armazena imagens (ex.: Docker Hub).
- Comunicação cliente ↔ daemon via **API**.

Imagem × Contêiner

- **Imagem:** modelo **somente leitura**, com a aplicação e dependências.
- **Contêiner:** instância **em execução** de uma imagem.
- Analogia: imagem = classe; contêiner = objeto.
- Uma imagem pode gerar **vários** contêineres.

Camadas e Union File System

- A imagem é formada por **camadas** sobrepostas.
- Cada instrução do Dockerfile cria uma **camada**.
- Camadas são **reaproveitadas** (cache) $\rightarrow$ builds rápidos.
- O contêiner adiciona uma camada **gravável** no topo.

Dockerfile

```
FROM python:3.12-slim
WORKDIR /app
COPY . .
RUN pip install -r requirements.txt
EXPOSE 8000
CMD ["python", "app.py"]
```

- `FROM` define a base; `RUN` executa no build; `CMD` define o comando de execução.

Comandos Essenciais

- `docker build -t minha-app .` — constrói a imagem.
- `docker run minha-app` — executa um contêiner.
- `docker ps` — lista contêineres em execução.
- `docker images` — lista imagens.
- `docker pull/ push` — baixa/envia imagens.

Volumes e Redes (visão)

- **Volumes:** persistência de dados fora do contêiner.
- **Redes:** comunicação entre contêineres (bridge, host, overlay).
- Contêineres são **efêmeros**: dados persistentes vão em volumes.
- Aula 34 detalha volumes e redes.

Registro de Imagens

- **Docker Hub**: registro público padrão.
- Registros **privados** para imagens internas.
- `docker pull` baixa; `docker push` publica.
- Versionamento por **tags** (ex.: `nginx:1.27`).

Dica

Use **imagens base enxutas** (slim/alpine) e fixe **versões** nas tags para builds reprodutíveis.

Resumo da Aula

- Docker tem **cliente, daemon e registro**.
- **Imagem** é o modelo; **contêiner** é a execução.
- Imagens são **camadas**; o Dockerfile as define.
- Comandos: **build, run, ps, images, pull/push**.

Atividade Rápida

- Diferencie **imagem** e **contêiner**.
- O que faz cada linha do **Dockerfile** de exemplo?
- Cite **três comandos** e o que fazem.

Referências

- Docker — *Documentação oficial*.
- Docker Hub.
- Material da aula 31.

Próxima Aula

- **Aula 33:** Prática — Docker 1: imagens e contêineres.